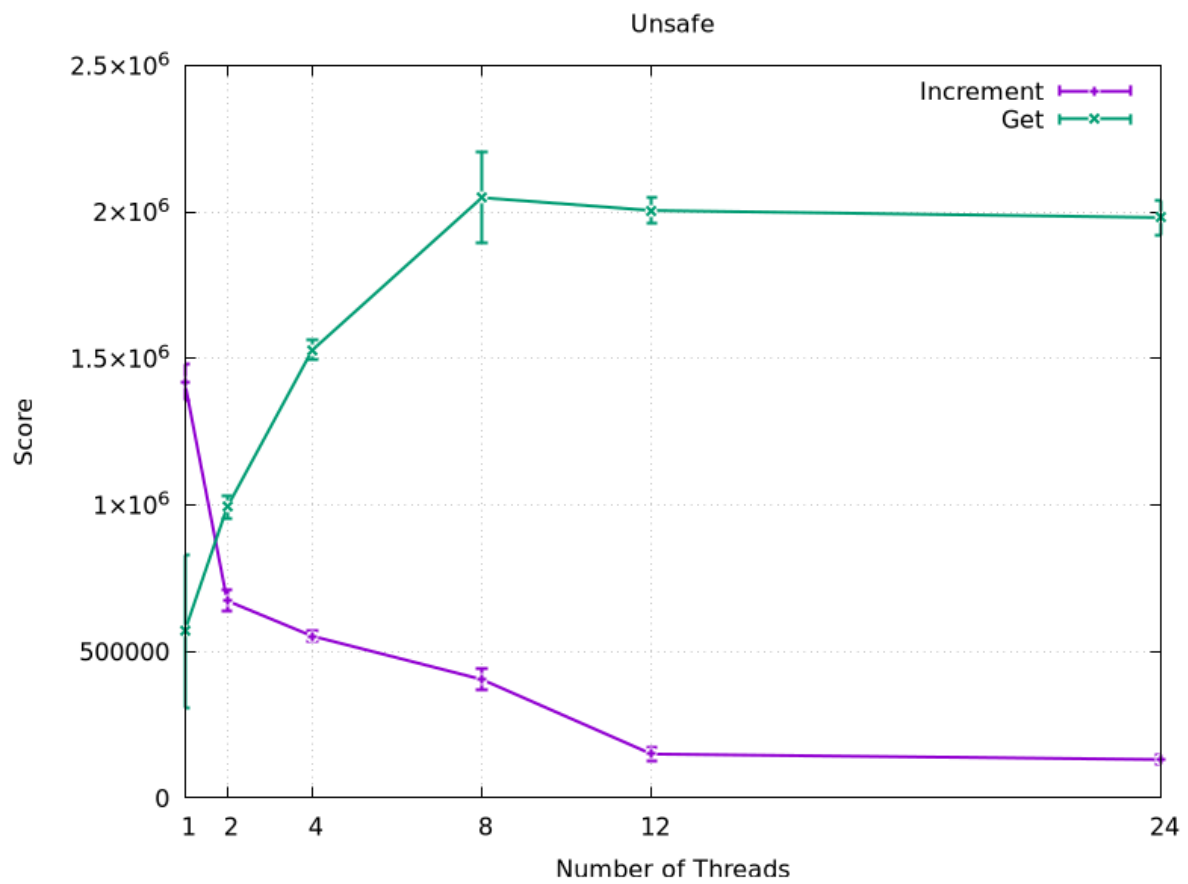


Easy

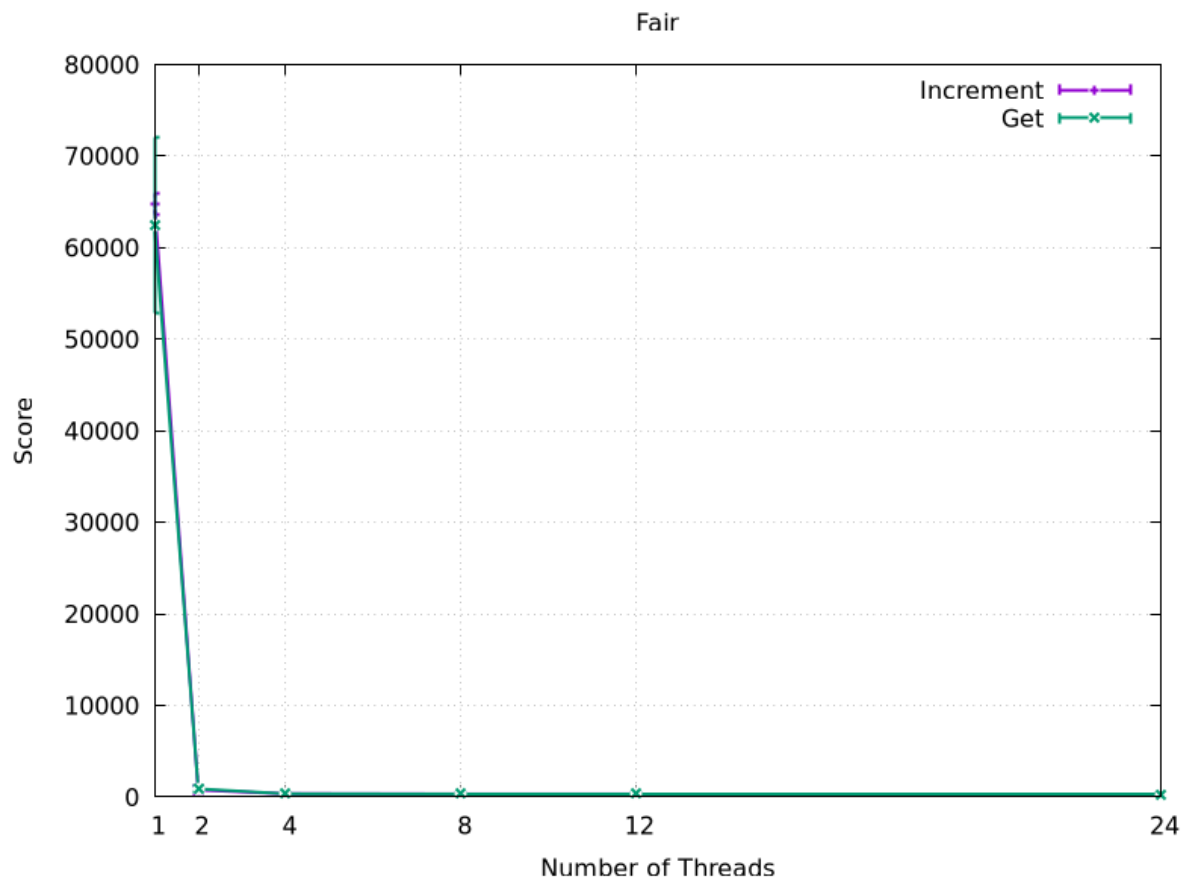
ThreadUnsafeCounter



- Compiler did not optimise my benchmarks out, they all return something.
- It does not scale well.
- In case of oversubscription it stales in it's performance.
- get seems to scale much better than increment.

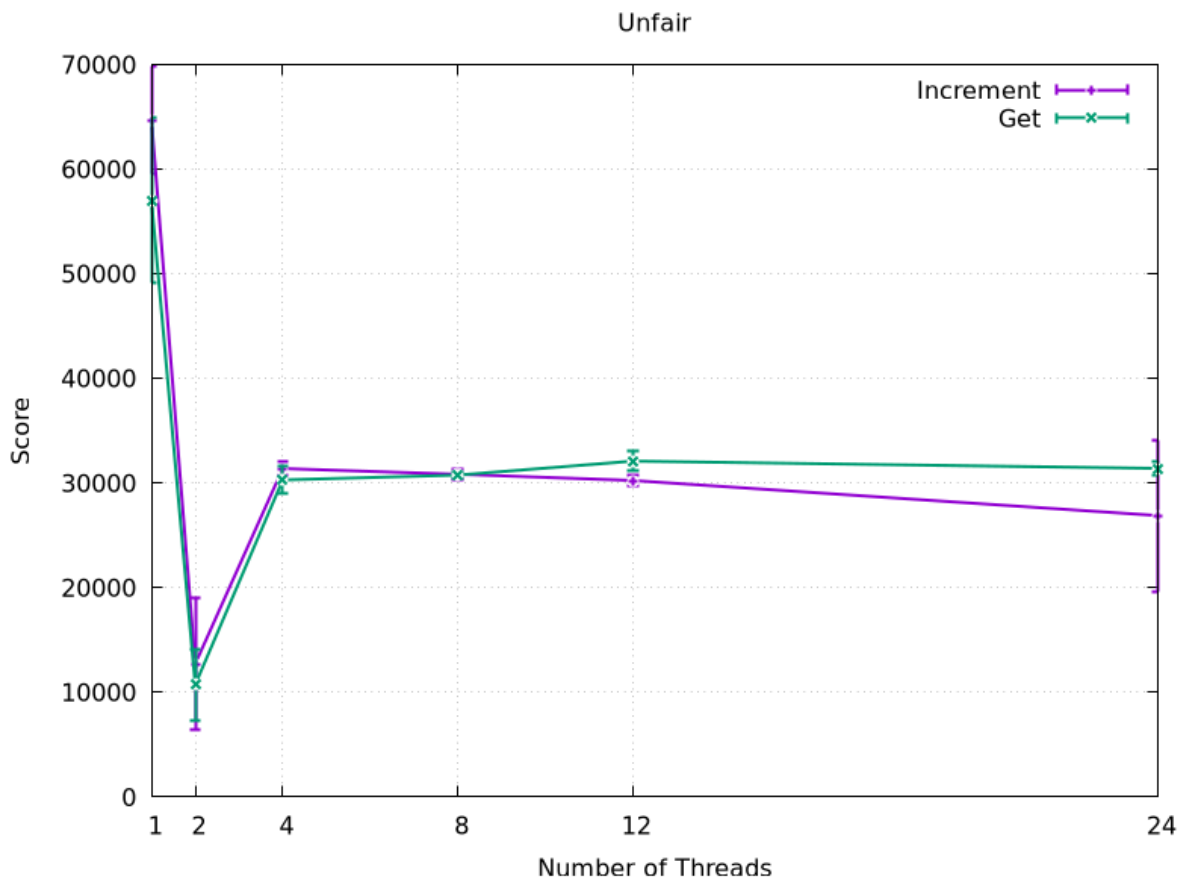
LockedCounter

Fair



- Does not scale at all. It has terrible performance costs for multiple threads, even 2.
- It works worse in case of oversubscription.

Unfair



- Scales well enough.
- Works well even in case of oversubscription.

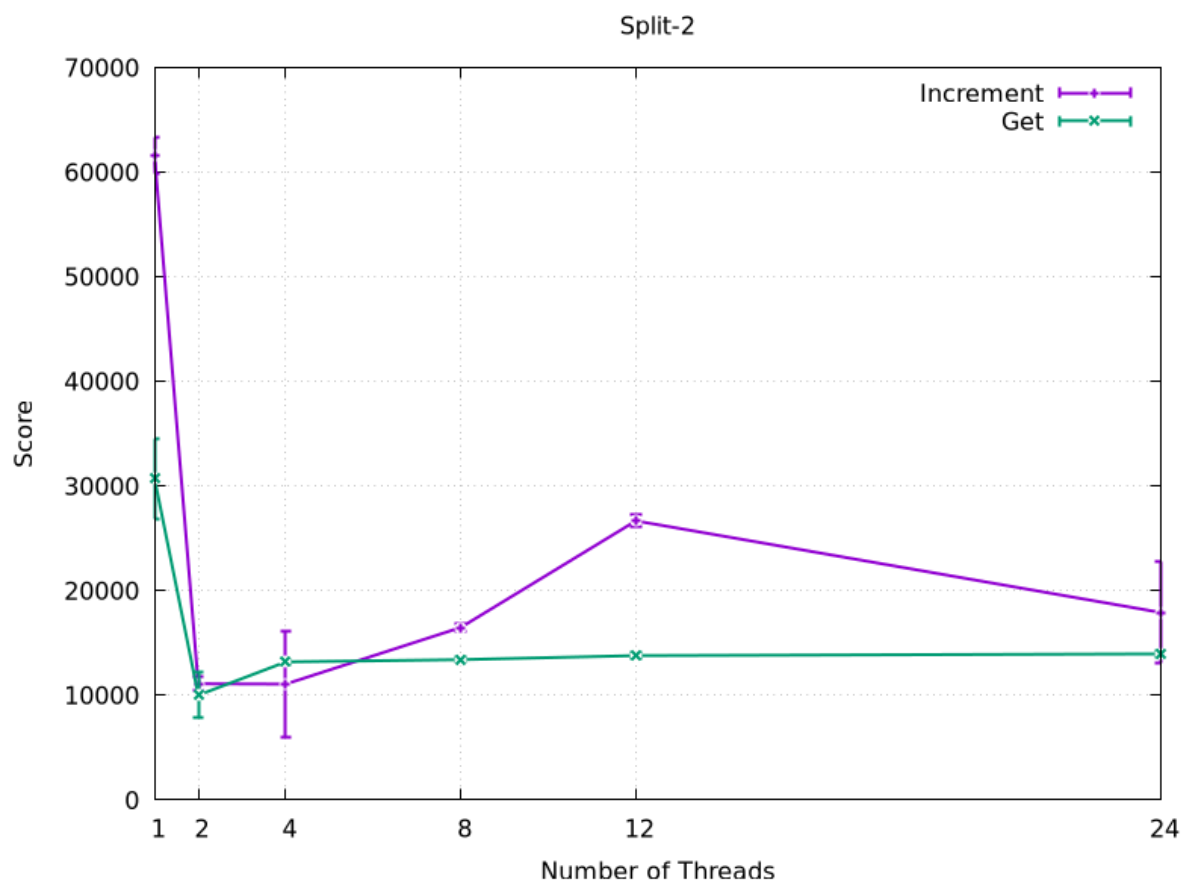
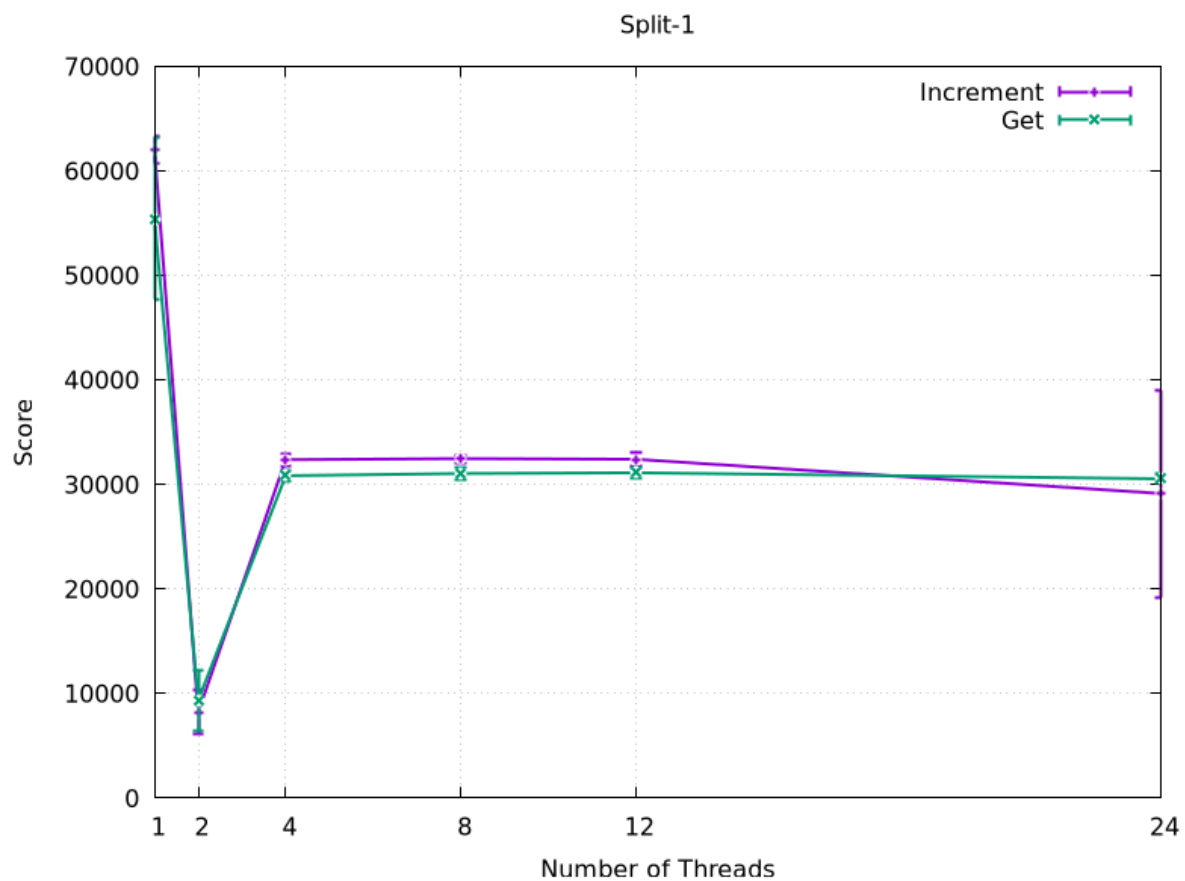
My opinions

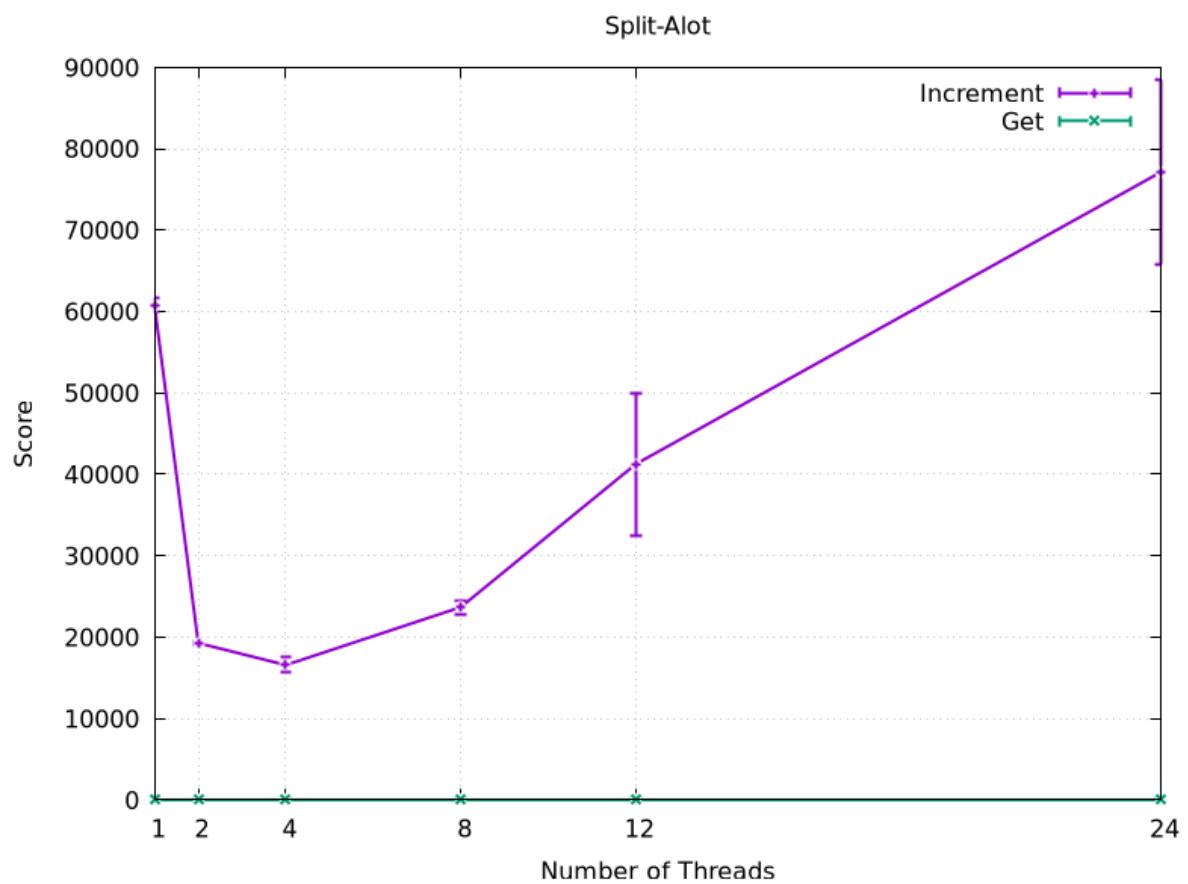
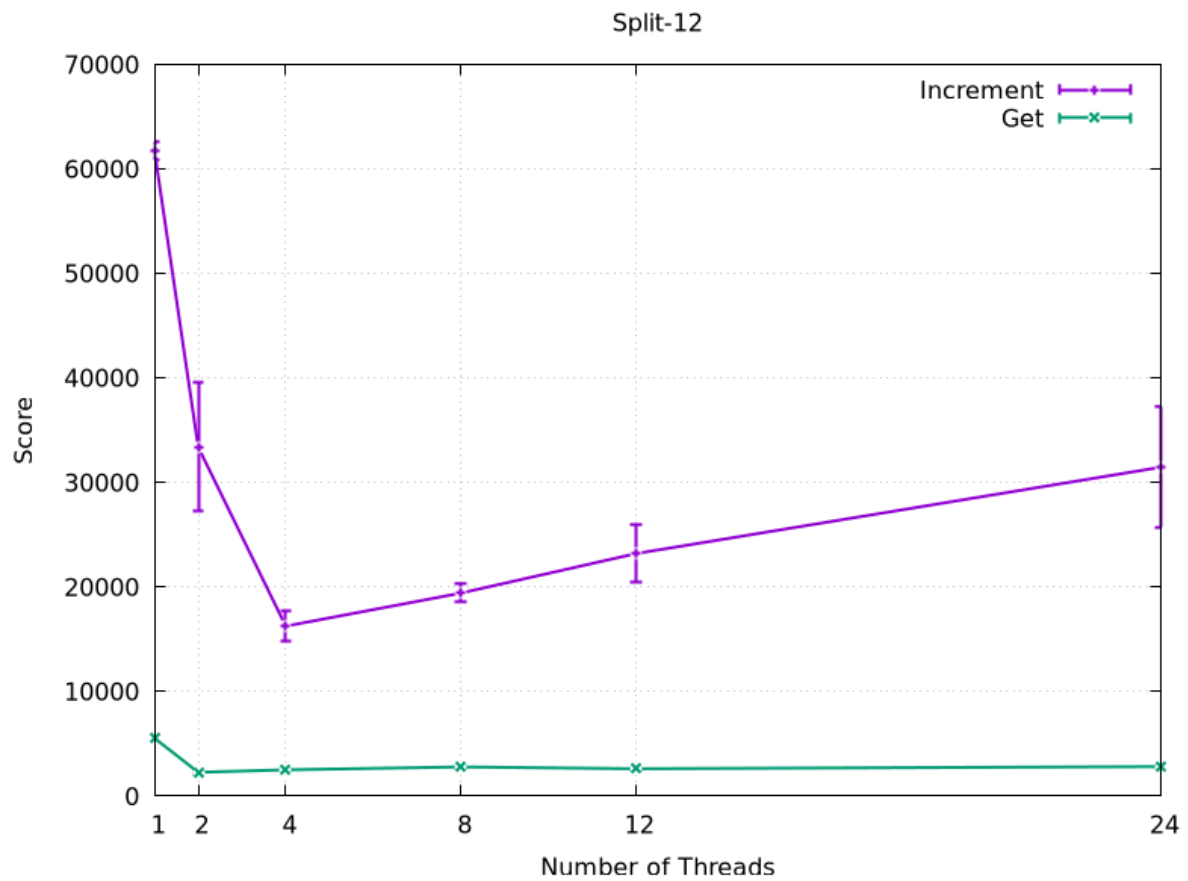
- Yes, there is, and it's quite big. My theory is, locking must be quite expensive of an operation. It does not depend on the operation much, since we lock in either.
- There is a huge gap. My reasoning is: In a fair lock, N Threads wait for the lock. i-th thread manages to do so, does it's thing, then releases the lock. The fairness will require the lock to be granted to a j-th thread, on the condition of $i \neq j$. This requirement has a cost - context switch. The doc says "The constructor for this class accepts an optional fairness parameter. When set true, under contention, locks favor granting access to the longest-waiting thread.". I assume that this means a context switch every time the lock is acquired (in a single operation context, the wait times may differ between locks if both get and increment are used in a benchmark.).

Medium

SplitCounter

- It does behave correctly and quite consistently. Errors are not so big on the graphs:





- It does scale well, while the number of threads is less than or equal to GRANULARITY.
- In case of oversubscription get does not slow down, since there are never more than GRANULARITY locks anyway. Increment slows down when the number of threads is bigger than GRANULARITY. It's better than with LockedCounter though, because we have less threads per slot in our case.
- get ultimately scales better relative to number of threads. We just don't have thread contention because just lock GRANULARITY locks, sum up, unlock, and we are fine. increment has a problem with thread contention whenever we have more threads than GRANULARITY. it becomes less predictable, i.e. dependant on the scheduler.
- Sure. Just use more threads than GRANULARITY. Our LockedCounter has the same issue. I assume the ways to alleviate this are to set GRANULARITY to number of system threads, or add locks on the fly. (e.g. we have a constraint GTON, which is $\frac{\text{GRANULARITY}}{N}$, where N is number of threads. When a thread uses us, we register it, the number of registered threads is effectively N . When GTON reaches certain limit, we add new locks.)